

Exercice 1:

- a) Si la taille n des vecteurs A, B et C est égale à 1500, de blocs de threads égale à 6.
- b) Avec la taille $n = 1500$, le nombre total de threads créés par ce lancement est 1536 (256×6)
- c) pour $n=1500$, la condition de if est nécessaire car on a besoin de 1500 thread hors le nombre de thread lancé égale à 1536. donc on a besoin de if pour n'accède pas au tableau hors limite.
- d) si on supprime la ligne if ($i < n$) et on remplace la configuration du kernel par $\langle n, 1 \rangle$, signifie qu'on crée un blocs avec 1 thread chacun dans notre cas 1500 blocs, cela réduire la performance car on n'applique pas le parallélisme convenablement.

Exercice 2:

a)

- **Initialisation et allocation de la mémoire (statique) sur CPU (host):**

```
float v1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
float v2[] = {1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9};
```

```
int memsize = sizeof(v1);
```

```
int vsize = memsize / sizeof(float);
```

```
float res[vsize];
```

- **Allocation de la mémoire sur GPU (device)**

```
float *Cv1;
```

```
cudaMalloc((void **)&Cv1, memsize);
```

```
float *Cv2;
```

```
cudaMalloc((void **)&Cv2, memsize);
```

```
float *Cres;
```

```
cudaMalloc((void **)&Cres, memsize);
```

- **Copie des données vers la mémoire GPU**

```
cudaMemcpy(Cv1, v1, memsize, cudaMemcpyHostToDevice);
```

```
cudaMemcpy(Cv2, v2, memsize, cudaMemcpyHostToDevice);
```

- **Appel du kernel**

```
multVect<<<2, vsize/2>>>(Cv1, Cv2, Cres);
```

- **Copie du résultat dans la mémoire CPU**

```
cudaMemcpy(res, Cres, memsize, cudaMemcpyDeviceToHost);
```

- **Affichage du résultat**

```
int i = 0;  
printf("res = { ");  
for (i = 0; i < vsize; i++) {  
    printf("%2.2f ", res[i]);  
}  
printf("\n");
```

- b)** le CPU et le GPU n'utilisent pas la même mémoire, il utilise des mémoires séparées physiquement. le GPU ne peut pas accéder à la mémoire RAM et ne partage pas les données automatiquement, donc on doit les transformer manuellement. Pour cette raison, on copie les données de cpu vers gpu avec cudaMemcpy le gpu fait appel à la fonction kernel pour faire le traitement nécessaire après on copie le résultat de gpu vers cpu pour qu'il affiche le résultat.